Backend-ul, etapă cu etapă

Spring Boot 4.1 · Java 21 · MySQL 8 · Gradle · Flavius

Lucrezi după două documente din `slot-mate-docs/`: `STRUCTURA_BD.pdf` = tabelele · `openapi.yaml` = contractul API-ului (îl deschizi în `editor.swagger.io`). Numele DTO-urilor = numele schemelor din `openapi.yaml`; numele metodelor din controllere = `operationId`. Fiecare etapă = un commit (sau mai multe) și se termină doar când trece „Gata când”.

0 Fundația

- Toolchain Java 21; scoate `restdocs` și `asciidoctor` din `build.gradle`.
- Mută testul în pachetul `ro.mycode.app`, lângă clasa `main`.
- MySQL 8 în `slot-mate-db/docker-compose.yml`; conexiunea din variabile de mediu: `DB_URL`, `DB_USER`, `DB_PASSWORD`, cu valori implicite pentru local.
- Flyway (`spring-boot-starter-flyway` + `flyway-mysql`): `V1__schema.sql` = cele 6 tabele din `STRUCTURA_BD.pdf`; `V2__categories.sql` = câteva categorii. `spring.jpa.hibernate.ddl-auto=validate`.
- Testele pornesc un MySQL în Testcontainers (`spring-boot-testcontainers`, `@ServiceConnection`).

Gata când: `./gradlew bootRun` pornește și Flyway aplică V1 și V2; `contextLoads` e verde.

1 Modelele (entitățile JPA)

- Pachete pe funcționalitate: `user`, `category`, `provider`, `offering`, `slot`, `booking`, `auth`, `common`.
- Câte o entitate pe tabel: `User`, `Category`, `Provider`, `Offering`, `Slot`, `Booking` + enum-ul `Role` (`@Enumerated(EnumType.STRING)`).
- Cheile străine: `@ManyToOne(fetch = LAZY)`; `Provider.user` și `Booking.slot` = `@OneToOne`. Relații doar pe partea care are cheia străină.
- Unicitatea pe pereche `(offering_id, start_time)`: în `@Table(uniqueConstraints = …)`, nu cu `unique = true` pe coloane.
- Lombok: `@Getter`, `@Setter`, `@NoArgsConstructor` — nu `@Data` pe entități.

Gata când: aplicația pornește cu `ddl-auto=validate` (entitățile se potrivesc cu schema).

2 Spring Data JPA (repository-urile)

- Câte un `JpaRepository` pe entitate, cu metodele de care au nevoie operațiile din `openapi.yaml`: `findByEmail`, `existsByEmail`, `findByUserId`, `findByProviderIdAndActiveTrue`, `existsByOfferingIdAndStartTime`, `existsBySlotId`…
- `@Query` pentru: căutarea furnizorilor cu filtre opționale (`categoryId`, `city`), intervalele libere ale unui serviciu între două date, rezervările unui client și ale unui furnizor.
- Câte un test `@DataJpaTest` (pe MySQL din Testcontainers) pentru fiecare `@Query`, plus unul care arată că a doua rezervare pe același `slot_id` aruncă `DataIntegrityViolationException`.

Gata când: testele de repository sunt verzi.

3 Securitatea

- `spring-boot-starter-security` + `spring-boot-starter-oauth2-resource-server`; JWT semnat HS256 cu `NimbusJwtEncoder` / `NimbusJwtDecoder`, valabil 3600 s, cu claim-ul `role` transformat în `ROLE_CLIENT` / `ROLE_PROVIDER` / `ROLE_ADMIN`.
- Cheia vine doar din variabila `JWT_SECRET` (minim 32 de caractere). În repo nu intră nicio cheie.
- Parolele cu `BCryptPasswordEncoder`. `register` nu acceptă `ADMIN`; email existent → 409. `login` greșit → 401, același mesaj pentru email inexistent și parolă greșită.
- `SecurityFilterChain` stateless, fără CSRF. Publice: `register`, `login`, GET pe categorii, furnizori și intervale, Swagger UI, `/v3/api-docs/**` și `/error`.
- `@EnableMethodSecurity`: rolul se verifică cu `@PreAuthorize("hasRole('PROVIDER')")`.

> **Gata când:** `register` → `login` → `GET /api/users/me` cu `Bearer` = 200; fără token = 401.

4 Serviciile și API-ul

- Pe rând: categorii → furnizori → servicii → intervale → rezervări, exact după `openapi.yaml`.
- DTO-uri ca `record`, validate cu `@Valid` și adnotările Jakarta (`@NotBlank`, `@Positive`, `@Email`…); conversia entitate ↔ DTO scrisă de mână.
- Proprietarul se verifică în service: un furnizor modifică doar ce e al lui, altfel 403.
- Reguli: `endTime` = `startTime` + durata serviciului; interval în trecut → 400; interval deja rezervat → 409 (și când îl prinzi din `DataIntegrityViolationException`).
- Erorile într-un singur `@RestControllerAdvice`, ca `ProblemDetail` (400 cu `errors` câmp → mesaj, 404, 409).
- `springdoc-openapi-starter-webmvc-ui` **3.1.x** (2.x nu merge pe Boot 4).

> **Gata când:** Swagger UI-ul aplicației (`/swagger-ui.html`) are aceleași 21 de operații ca `openapi.yaml`, iar fluxul complet merge din el.

5 Testarea

- Unit, cu Mockito: regulile din service-uri (rezervare, anulare, proprietar).
- `@WebMvcTest`: validarea (400) și forma `ProblemDetail`.
- Integrare, `@SpringBootTest` + MockMvc + Testcontainers, **prin login real** (nu doar `@WithMockUser`): furnizor se înregistrează → profil → serviciu → interval; client se înregistrează → rezervă (201) → a doua rezervare pe același interval (409) → anulează (204) → rezervă din nou (201).
- Accesul: fără token → 401; client pe `POST /api/offerings` → 403; furnizor pe serviciul altui furnizor → 403.
- Structura fiecărui test: `// Arrange`, `// Act`, `// Assert`.

> **Gata când:** `./gradlew test` e verde pe un calculator cu Docker pornit și nimic altceva instalat.

> **Spring Boot 4 față de exemplele de pe internet (scrise pe Boot 3):** `@MockBean` → `@MockitoBean` · `@WebMvcTest` din `org.springframework.boot.webmvc.test.autoconfigure` · `@DataJpaTest` din `org.springframework.boot.data.jpa.test.autoconfigure` · Jackson 3: `tools.jackson.databind.ObjectMapper` · starter-ele de test sunt separate: `…-webmvc-test`, `…-data-jpa-test`.